

A1.1

Computer hardware and operation

What principles underpin the operation of a computer, from low-level hardware functionality to operating systems' interaction?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A1.1.1 Describe the function and interaction of the main central processing unit (CPU) components
- ▶ A1.1.2 Describe the role of a graphics processing unit (GPU)
- ▶ A1.1.3 Explain the difference between the CPU and the GPU (HL)
- ▶ A1.1.4 Explain the purposes of different primary memory types
- ▶ A1.1.5 Describe the fetch, decode and execute cycle
- ▶ A1.1.6 Describe the process of pipelining in multi-core architectures (HL)
- ▶ A1.1.7 Describe the internal and external types of secondary memory storage
- ▶ A1.1.8 Describe the concept of compression
- ▶ A1.1.9 Describe the different types of services in cloud implementation



■ A central processing unit (CPU) from above and underneath

A1.1.1 Function and interaction of the main central processing unit components

■ What is the central processing unit?

The central processing unit (CPU) is often referred to as the “brain” of the computer. It is a critical component that carries out the majority of the processing inside a device.

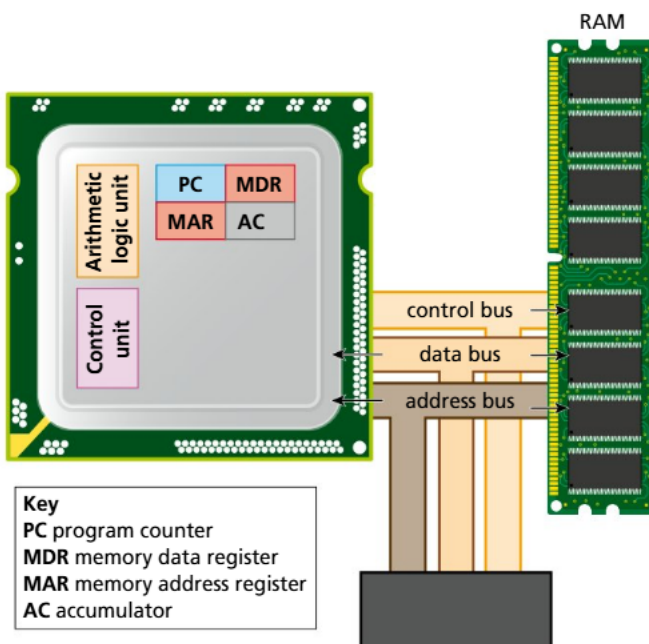
The CPU is made up of two main units: the control unit (CU) and the arithmetic logic unit (ALU).

Control unit (CU)

The control unit directs the operations of the processor. It is responsible for the fetch–decode–execute cycle, managing all three operations and directing the computer's memory, ALU and input/output devices to respond appropriately.

Arithmetic logic unit (ALU)

This unit is responsible for performing arithmetic and logic operations. These include basic arithmetic operations such as addition, subtraction, multiplication and division, as well as logic operations including AND, OR, XOR and NOT.



■ A model of the CPU

■ What are registers?

Registers are very small amounts of storage that are available directly on the CPU to hold temporary data that the CPU may be working on. The registers are instruction register (IR), program counter (PC), memory address register (MAR), memory data register (MDR) and accumulator (AC).

Instruction register

When an instruction is fetched from memory, it is held in the IR within the CPU. This register holds the instruction that is currently being executed by the CPU.

Program counter

The PC holds the address of the next instruction that is to be fetched from memory. Once the instruction has been fetched, the PC updates to point to the next instruction that will be needed.

Memory address register

The MAR holds the memory address that is currently being fetched. The content from the PC is copied to the MAR, and the MAR provides this address to the memory unit, so that data and instructions can be read from or copied to that location.

Memory data register

This holds the data that has been fetched or is about to be written to the memory address currently in the MAR.

Accumulator (AC)

This stores the intermediate arithmetic or logical results produced by the ALU.

■ What are buses?

Buses are a critical component of the computer system, as they transfer data between various devices, including the CPU, memory, storage and peripherals. Buses have *widths* that are measured in bits. The bigger the width of the bus, the more data it can transmit at one time. There are three main types of buses: control bus, data bus and address bus.

Control bus

The control bus is used to transmit command and control signals from the CPU to other components of the system, and vice versa. Due to the need for signals to be sent and received, this bus is **bidirectional**. Some of the signals that would be transmitted via the control bus are read / write operations, interrupt requests, clock signals for synchronization and status signals from hardware components.

◆ **Bidirectional bus:**
a bus that can transfer data in both directions.

Data bus

The data bus carries the data being processed between the CPU, memory and other peripherals. The width of the data bus is important for determining the amount of data it can transfer at one time. Common data bus widths are 8, 16, 32 and 64 bits. As data needs to be read from and written to memory, data buses are usually bidirectional.

Address bus

The address bus is used to transmit the address that is to be read from or written to in memory. The width of this bus determines the memory capacity of the system. For example, a 32-bit address bus can address 2^{32} memory locations.

■ What are cores?

CPUs come in a number of different configurations. These include single-core processors, multi-core processors and co-processors.

Single-core processors

This CPU has a single processing unit, meaning it can only handle one task at a time. These are more often found in low-end computers or older machines. They are adequate for simple tasks that do not require heavy multitasking. Single-core processors are able to run more than a single application at a time, but the CPU has to be shared between these applications, which can impact performance.

Multi-core processors

A CPU with multi-core processors has two or more cores that can run multiple instructions simultaneously. These are often referred to as dual-core (two processors), quad-core (four processors), hexa-core (six) or octa-core (eight). Their performance is significantly faster than single-core processors and they are ideal for multitasking, gaming and servers. However, software has to be written to take advantage of these extra cores. Older software that does not do this would likely run at a similar speed as on a single-core processor.

Co-processors

A co-processor is a special type of processor that has a specific job to support the main CPU. These are built with a distinct purpose to achieve optimal performance compared to a general-purpose CPU. Tasks are offloaded by the CPU to the co-processor so they can run in parallel, enhancing the system's performance. Examples of co-processors are graphics processing units (covered in Section A1.1.2), audio processors and digital signal processors (DSPs), which are used in telecommunications and image compression.

● Common mistake

A common mistake is thinking that adding more cores to a CPU always makes it faster in a straightforward way – like assuming a dual-core CPU is twice as fast as a single-core, or a quad-core is four times faster. This isn't always true, because the speed increase depends on how well the software can use multiple cores at the same time. Many programs aren't designed to take full advantage of multiple cores, so the extra cores may not make a noticeable difference. Other factors, such as memory speed and how the CPU is designed, also affect how fast it can run. So, just having more cores doesn't automatically mean much faster processing.

REVIEW QUESTIONS

- 1 What is the primary function of the arithmetic logic unit (ALU) in a computer's CPU?
- 2 How does the control unit (CU) direct the operations of the CPU?
- 3 Why is the program counter (PC) important for executing a sequence of instructions?
- 4 What roles do the data bus and address bus play in the functioning of the CPU?
- 5 How does the memory address register (MAR) work in conjunction with other CPU components to access memory?
- 6 How do multi-core processors differ from single-core processors in handling tasks?



■ A graphics processing unit (GPU)

A1.1.2 Role of a graphics processing unit

A graphics processing unit (GPU) is a specialized electronic circuit designed to accelerate the rendering of images, videos and animations by performing rapid mathematical calculations. Initially developed to handle the demanding graphics workloads of video games and visual applications, GPUs have evolved to play a crucial role in various fields beyond graphics rendering. Their structure, consisting of thousands of small, efficient cores, allows them to process multiple tasks simultaneously, making them exceptionally well-suited for computationally intensive applications. This capability has led to their widespread adoption in scientific research, machine learning, artificial intelligence and cryptocurrency mining. By offloading these intensive tasks from the CPU, GPUs enhance overall system performance, enabling faster and more efficient data processing and visualization.

■ Graphics processing



■ Video game graphics

GPUs are designed with a highly parallel structure, enabling them to perform many calculations simultaneously. This makes them exceptionally well-suited for rendering the complex and resource-intensive graphics seen in modern video games and applications. They also handle the application of **shaders and textures** to 3D models, which includes lighting, shading and texture mapping, enhancing the realism of the scene.

■ Video processing

GPUs assist in the decoding and encoding of video files, making processes such as playback, streaming and editing more efficient and faster. This is particularly helpful for those working with high-resolution video files of 4k or higher.

■ Artificial intelligence and machine learning

GPUs were originally created for graphical processing; however, in the early 2000s, researchers and engineers began to recognize their potential for handling general-purpose calculations, including those required for machine learning and AI. The shift towards using GPUs for this was largely due to their ability to perform many simple calculations simultaneously, and because many GPUs can be run in **parallel**. Many AI models rely heavily on **matrix and vector multiplications**, and GPUs far outperform a CPU when trying to process these quickly.

◆ **Shaders and textures:** techniques used in 3D rendering to apply effects, lighting and details to models.

◆ **Parallel processing:** the ability of the GPU to perform many calculations simultaneously due to its highly parallel structure.

◆ **Matrix and vector multiplications:** fundamental operations in machine learning and graphics that involve complex mathematical calculations.

◆ **Deep learning:** a subset of machine learning that uses an artificial neural network to imitate the design of the human brain to find generalizations in complex data that can be used for decision-making.

◆ **Proof of work:** a consensus mechanism requiring cryptominers to solve complex problems to add a new block to the blockchain.

This realization gained momentum as machine learning models, especially **deep learning** models, became more complex and required significant computational power for training. By the mid-2000s, GPUs had become essential tools in the field of AI and machine learning, transforming how data scientists and researchers approached problems, significantly reducing the time it took to train complex models.

■ Blockchain and cryptocurrency mining



■ The cryptocurrency boom – at its peak in November 2021, the total market capitalization of cryptocurrencies reached approximately \$3 trillion



In 2010, the use of GPUs for Bitcoin mining surged as miners discovered that GPUs significantly outperformed CPUs in solving cryptographic puzzles, such as finding the nonce in the hashing algorithm for the **proof-of-work** system. This realization led to a dramatic shift towards GPU mining. The cryptocurrency boom between 2017 and 2021 further escalated the demand for GPUs, resulting in skyrocketing prices and global shortages.

As of 2023, this demand had reduced somewhat and prices of GPUs were becoming more stable. This was for a number of reasons:

- The volatility and reduced profitability of cryptocurrency mining had led to less demand for GPUs, specifically for mining purposes.
- Big manufacturers had increased production to meet the demands.
- Application-Specific Integrated Circuits (ASICs), which are specifically designed for mining, had largely replaced the use of GPUs in many mining operations.

REVIEW QUESTIONS

- 1 What is the role of a graphics processing unit (GPU) in a computer?
- 2 How do GPUs enhance the performance of video games and video processing tasks?
- 3 Why have GPUs become essential in fields such as artificial intelligence, machine learning and cryptocurrency mining?

A1.1.3 Differences between the CPU and the GPU (HL)

The central processing unit (CPU) and the graphical processing unit (GPU) are both core components of modern computers. They are designed differently, which is why they are used for different kinds of tasks. The CPU is great for handling various jobs, but the GPU is better for doing the same job many times on a lot of data at once.

■ Design philosophies

CPUs are generally called “general-purpose processors” because they can handle many types of tasks. They are designed to run the operating system, process user input and manage programs. CPUs are good at tasks where decisions need to be made quickly, and where different types of work are being done at the same time.

GPUs are specialized processors because they focus on specific types of tasks. They are made for processing large amounts of data in parallel. This means they can work on many calculations at the same time. For example, GPUs are used to process images and videos because they can work on thousands of pixels at once.

■ Core architecture

The CPU has only a few cores, but these cores are very powerful. Each core can handle many different instructions, but it works best when doing one task at a time. This makes the CPU very good for such tasks as running the operating system, where quick responses are needed. CPUs also have features including branch prediction (where the CPU tries to guess what will happen next) and out-of-order execution (where the CPU can work on tasks that are ready before others).

The GPU has many smaller cores. These cores are not as powerful as the CPU cores, but there are thousands of them, and they all work at the same time. This is why the GPU is very good for tasks such as rendering 3D images, where many similar calculations need to happen at once. The GPU's architecture is designed to work on large sets of data all at the same time.

■ Memory access and power efficiency

The CPU and GPU access memory differently. The CPU uses a smaller, high-speed memory cache to get data quickly. This is useful when the CPU needs to access small amounts of data many times, such as when running programs or handling user inputs.

The GPU uses its own special memory called VRAM (video RAM). VRAM has a very high bandwidth, meaning it can move large amounts of data at once, such as images and videos. However, the GPU uses more power because it must process a lot of data at the same time, especially when rendering videos or running complex simulations.

■ Comparison of central processing units (CPUs) and graphics processing units (GPUs)

Processor	Processing	Architecture	Functionality
CPU	It is a general purpose processor, capable of handling many different tasks. It executes the instructions of computer programs, involving operations such as arithmetic, logic and controlling input / output (I/O) operations, as directed by the operating system.	CPUs generally have fewer cores. General user devices tend to have between 4 and 8 cores; however, there are some advanced CPUs that now have 64 cores or more. Each core is very versatile, making it capable of handling complex computations that require sequential processing.	Allows the user to switch between multiple tasks and applications. This makes it ideal for running the operating system and general software applications.
GPU	It is a specialized processor, with a focus on handling graphics, rendering images, video and animations.	Composed of hundreds or thousands of small cores that are well-suited for tasks that can be run in parallel. While each core is not as powerful as a standard CPU core, the high number of cores allows them to perform a large number of calculations simultaneously, making them perfect for graphical processing.	Suited for tasks that require simultaneous processing of large blocks of data, such as rendering images, video processing and deep learning applications.

◆ **Rendering:** the process of generating an image from a model by means of computer programs.

Key information

To summarize, CPUs are better for tasks that require high-speed, complex decision-making and versatility. GPUs are better when the same operation needs to be performed on many data points simultaneously. This means that for tasks such as gaming, video editing and computational research (AI and machine learning), GPUs often significantly outperform CPUs.

How the CPU and GPU work together to increase video-game performance

When playing video games, the CPU and GPU work together to deliver a seamless and immersive experience. The CPU handles the game's core logic, including rules, physical calculations and AI behaviour. It processes the inputs from the player (processing the outcomes of their actions and updating the game state accordingly). The GPU's primary role is to render the game's visuals. It processes **vertex and pixel data** to draw images on to the screen, including 3D objects, textures and effects such as lighting and shadows.

◆ Vertex and pixel

data: data used by the GPU to render 3D objects and images.

◆ **Frame:** a single image in a sequence of images that makes up a video or animation.

PROGRAMMING EXERCISE

Run benchmark software on your device to see your overall system performance. There are many options out there that you can search for; <https://novabench.com> and www.userbenchmark.com have free versions.

Typical scenario

- 1 **Player input:** The player presses a key to move a character. The CPU processes this input, updates the character's position based on game physics and determines the new game state.
- 2 **Data preparation:** The CPU prepares the new position and state data and sends it to the GPU.
- 3 **Rendering:** The GPU updates the **frame** with the character's new position, applies lighting and shading and renders the scene.
- 4 **Display:** The rendered frame is displayed on the screen, providing immediate feedback to the player.

REVIEW QUESTIONS

- 1 How do the CPU and GPU work together to enhance video-game performance?
- 2 Why is a GPU better suited than a CPU for tasks such as video rendering or AI computations?
- 3 What are shaders and textures, and how do they contribute to the rendering process handled by the GPU?

A1.1.4 Purposes of different primary memory types

Memory types

The primary memory of the computer stores data and instructions that the CPU needs in order to process tasks. Primary memory includes several different types: RAM (random access memory), ROM (read-only memory), caches and registers (covered in Section A1.1.1). These are all types of primary memory, meaning they are used directly by the CPU.



■ Sticks of random access memory (RAM)



■ Read-only memory (ROM) attached to a motherboard

◆ **Volatile:** a type of memory or storage that loses its data when the power is turned off.

RAM

RAM (random access memory) holds instructions and data for programs that are currently running. For example, when you open an app on your phone or computer, it loads into RAM so that it can be accessed quickly by the CPU.

RAM is **volatile**, meaning that it loses its contents when the power to the computer is turned off. This is why, when playing a game, you lose your progress unless you save the game (which is then stored in secondary memory).

One real-world example of using RAM is in smartphones, which use RAM to switch quickly between apps. When you leave an app, it stays in the RAM, so you can return to it quickly without reloading it from scratch.

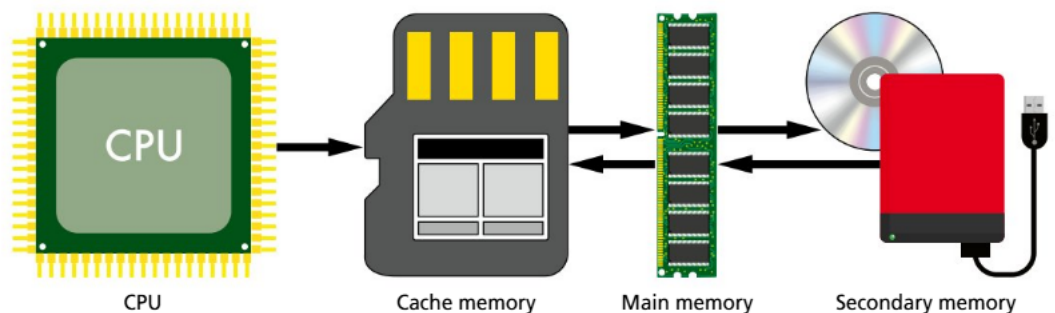
ROM

ROM (read-only memory) is used for storing instructions that are very rarely modified. ROM is used for the BIOS (basic input / output system) of the computer, which is located on the motherboard. The BIOS' main role is to initialize and test the system hardware components on startup, and to load the operating system (OS) software from the secondary memory storage into the RAM, ready for the CPU to fetch, decode and execute the instructions.

ROM is non-volatile memory, meaning it does not lose its contents when the computer does not have power. While ROM is "read only", meaning it cannot easily change its data, most modern computers use flash memory, which allows for updates and reprogramming. This allows motherboard companies to update their software when required.

A real-world example of using ROM is in smartphones, where ROM stores the operating system and core applications, which do not change unless you perform an update. This ensures that your phone can boot up reliably every time.

Cache (L1, L2 and L3)



■ The order a CPU goes through when trying to retrieve data

Cache memory is small, but provides high-speed access to the CPU compared to the RAM. It acts as a buffer between the CPU and the slower RAM, storing frequently used data and instructions.

There are three types of cache: L1, L2 and L3, each with different sizes and speeds. The closer to the CPU, the faster it is.

- **L1 cache** is located directly on the CPU, making it the fastest type of cache. It can be accessed almost instantly due to its location. However, it is also the smallest, often only a few kilobytes in size (32KB to 128KB per core). Each CPU core usually has its own L1 cache, which is typically split into two sections: L1i to store instructions and L1d to store data.
- **L2 cache** can either be on the CPU, like L1, or situated very close to the CPU. L2 cache is larger than L1 and can be up to several megabytes in size (256KB to 2MB per core), providing more storage for frequently used instructions. It is faster than L3, but slightly slower than L1, though it still significantly speeds up processing by reducing the need to fetch data from the slower RAM.
- **L3 cache** is often located the furthest from the CPU chip. L3 cache may be shared on multiple-core CPUs, whereas L1 and L2 are usually exclusive to a single core. It is the largest of the three, and can be up to tens of megabytes in size (2MB to 64MB shared across all cores). It is the slowest of the three types of caches, but is still significantly faster than RAM.

◆ **Cache hit:** when the CPU requests data and it is found in the cache memory.

◆ **Cache miss:** when the CPU requests data and it is not found in the cache memory, necessitating retrieval from slower main memory or storage.

The terms **cache hit** and **cache miss** are used to describe the efficiency of the CPU's cache memory when retrieving data. A cache hit is the ideal scenario, where the CPU requests data and it is found in the cache memory. A cache miss means it was not found, necessitating retrieval from the slower main memory (RAM) or even slower storage (SSD / HDD).

The percentage of hit rate determines the efficiency and effectiveness of the cache. A low percentage means the system would suffer more from latency, where the data has to be fetched from elsewhere, hindering performance speed. Systems with a larger cache size will generally perform better, as well as systems with more intelligent prefetching techniques that can predict which data will be needed soon and load it into cache ahead of time.

Top tip!

Imagine an onion with its layers representing the levels of cache:

- **L1 cache** is the smallest and fastest, like the very centre of the onion, where everything is tightly packed and closest to the core of the CPU.
- **L2 cache** is slightly larger and slower, like the next layer out – still close to the centre, but not as quick to access as the very core.
- **L3 cache** is the largest and slowest, like the outer layers of the onion. It's still important, but it takes a bit longer to get to, just like how the CPU takes a bit more time to access data in L3 cache compared to L1 and L2.

Optimizing CPU performance with cache

The cache plays a critical role in ensuring that the CPU can access data as quickly as possible. When the CPU finds the searched-for data in the cache (a cache hit), the data can be processed very quickly. However, when there is a cache miss, the CPU has to look for the data in the slower memory, which causes a delay.

Imagine you are playing a video game on a computer. The CPU frequently checks the L1, L2 and L3 cache to find the data it needs to run the game smoothly. The game's core functions, such as player controls and game logic, might be stored in the L1 cache, while the less frequently accessed data, such as background textures, may be in the L3 cache. The layering system helps to ensure that the game runs smoothly, without interruptions.

A CPU with a larger cache or more advanced prefetching (a technique where the CPU predicts what data it will need and loads it into cache ahead of time) has fewer cache misses and performs better overall.

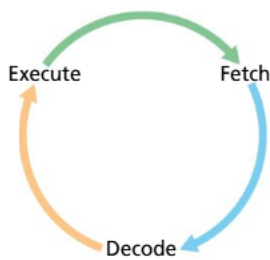
REVIEW QUESTIONS

- 1 What is the main purpose of RAM in a computer system, and why is it considered volatile?
- 2 How does ROM differ from RAM in terms of its function and volatility?
- 3 Why is cache memory important for CPU performance, and how do the different levels of cache memory (L1, L2, L3) vary in terms of speed and size?
- 4 What happens during a cache hit and a cache miss, and how do these events impact system performance?

A1.1.5 The fetch–decode–execute cycle

The fetch–decode–execute cycle, also known as the “instruction cycle”, is the fundamental process that a CPU uses to execute instructions. The cycle consists of three main stages:

- 1 **Fetch:** The CPU fetches an instruction from the memory.
- 2 **Decode:** The CPU interprets the instruction and prepares the necessary operations to execute it.
- 3 **Execute:** The CPU performs the actions required by the instruction.



■ The fetch–decode–execute cycle

■ Little Man Computer

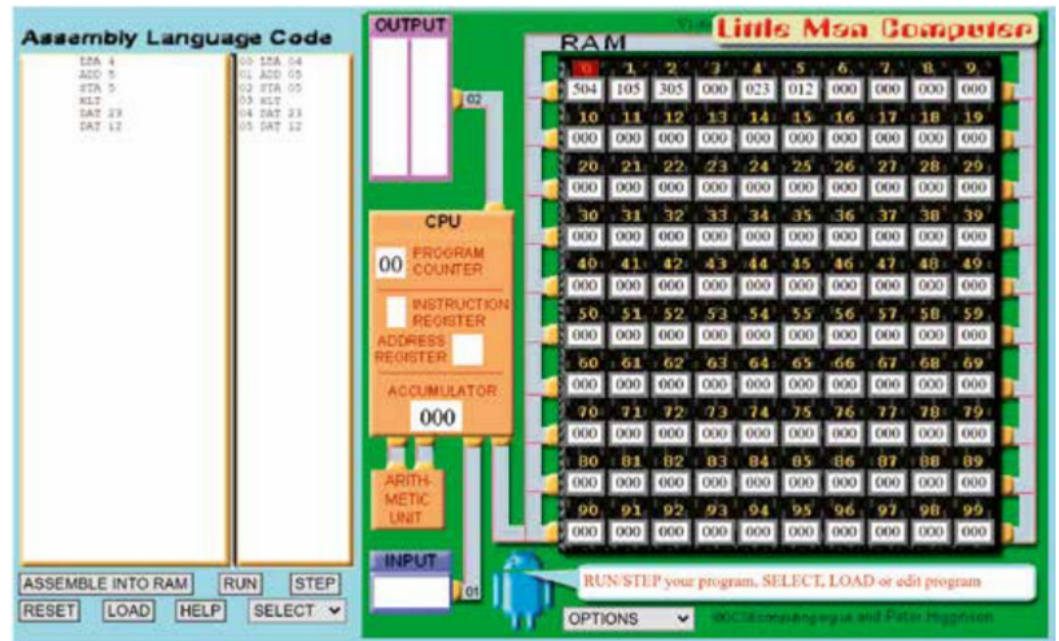
An easier way to see these stages carried out in more detail is to use an educational CPU model known as Little Man Computer, which you can search for online or use the one available here: <https://peterhigginson.co.uk/lmc>. This model uses assembly language – a simple set of instructions, each represented by three letters, which is stored as a three-digit code in the memory. The full set of instructions is:

Instruction	Code	Description
INP	901	Input a value and store it in the accumulator
OUT	902	Output the value from the accumulator
DAT	N/A	Used to define data values directly in memory at the point of declaration, often for constants or variables.
LDA	5XX	Load the value from the specified memory address into the accumulator
STA	3XX	Store the value in the accumulator at the specified memory address
ADD	1XX	Add the value from the specified memory address to the accumulator
SUB	2XX	Subtract the value from the specified memory address from the accumulator
HLT	000	Halt the program
BRA	6XX	Branch (jump) to the specified memory address
BRZ	7XX	Branch to the specified memory address if the accumulator is zero
BRP	8XX	Branch to the specified memory address if the accumulator is positive

Enter the following program into the left-hand column and assemble into RAM. You will see the three-digit representation for each instruction stored at a memory address on the right. For example, LDA 4 has been stored as 504 in memory address 0.

```
LDA 4
ADD 5
STA 5
HLT
DAT 23
DAT 12
```


Your LMC should look like this:



■ Peter Higginson's LMC model

First cycle

Click **step**.

- 1 **Fetch:** The PC (program counter) is currently set to 0, so the instruction at memory location 0 is fetched (504) by opening the 0 address in RAM using the address bus and fetching the instruction on the data bus. The control bus sends a read signal to initiate this process. 5 is stored in the instruction register and 04 in the address register. While this happens, you will see the PC gets incremented to 1 via the ALU, ready for the next instruction.
- 2 **Decode:** Once the instruction is fetched, the CPU decodes the instruction. The control unit uses the control bus to co-ordinate this process. The instruction stored in the instruction register is 5, which decodes as "load into the accumulator". The address register 04 indicates the address of the data to load.
- 3 **Execute:** The command is then carried out. Address 4 is opened on the address bus, and the control bus sends the appropriate signals to retrieve the data (23) from that location on the data bus and store it into the accumulator.

Second cycle

Click **step**.

- 1 **Fetch:** The CPU now uses the PC to know which instruction to fetch next: 1 is currently stored. Address 1 is opened, and the instruction 105 is fetched. The control bus sends a read signal to initiate this. 1 is stored in the instruction register and 05 in the address register. The PC is incremented to 2 by the ALU.
- 2 **Decode:** The instruction 1 is decoded as "add to accumulator"; the address register is the address of the data to add (5). The control unit uses the control bus to co-ordinate this.
- 3 **Execute:** Address 5 is opened, the data 12 is fetched and both the accumulator (currently 23) and the fetched data (12) are passed to the ALU. The result of $23 + 12$ is stored in the accumulator (35).

Third cycle

Click **step**.

- 1 **Fetch:** The PC is currently 2, so the instruction at memory address 2 is fetched (305). The control bus sends a read signal to initiate this. 3 is stored in the instruction register, and 05 is stored in the address register.
The PC is incremented to 3 via the ALU.
- 2 **Decode:** The instruction 3 decodes as “store accumulator to address” and the address register gives the location of where to store the data (05). The control unit uses the control bus to co-ordinate this.
- 3 **Execute:** Memory address 5 is opened via the address bus, and the control bus sends the appropriate signals to send the accumulator contents down the data bus and store them at address 5 (overwriting the current data).

Fourth cycle

Click **step**.

- 1 **Fetch:** The PC is currently 3, so the instruction at memory address 3 is fetched (000). The control bus sends a read signal to initiate this. 0 is stored in the instruction register, and 00 is stored in the address register.
The PC is incremented to 4 via the ALU.
- 2 **Decode:** The instruction 0 decodes as “halt”. The control unit uses the control bus to signal this operation.
- 3 **Execute:** The computer halts all operations and ends the program.



Common mistake

A common mistake is assuming that the program counter (PC) gets updated after the execute stage of the fetch–decode–execute cycle. The PC is usually updated during or immediately after the fetch stage, so it points to the next instruction in memory before the current instruction is even decoded or executed. This ensures that the CPU always knows where to find the next instruction in the sequence.

PROGRAMMING EXERCISES

Write an LMC program to:

- 1 input two numbers, add them, and output the result
- 2 input a number and output whether it is positive or zero
- 3 calculate the sum of the first five natural numbers
- 4 input two numbers and output the larger one
- 5 input three numbers and output them in ascending order.

REVIEW QUESTIONS

- 1 What are the main steps in the fetch–decode–execute cycle, and why is this cycle fundamental to CPU operations?
- 2 How does the CPU use the address, data and control buses during the fetch–decode–execute cycle?
- 3 Why is the interaction between memory and registers crucial during the fetch phase of the CPU cycle?

A1.1.6 The process of pipelining in multi-core architectures (HL)

◆ **Multi-core architectures:** systems with multiple CPU cores on a single chip, allowing parallel execution of instructions and tasks.

Pipelining is a powerful technique used in **multi-core architectures** to enhance CPU performance by overlapping the execution of multiple instructions. To understand this concept, imagine a carwash service that processes cars through several stages: initial wash, detailed cleaning, rinse and drying. Each stage takes five minutes.



■ A carwash team operating in parallel execution to get the job done faster

In a non-pipelined operation, each car must complete all stages before the next car begins:

Car								
A	initial wash	detailed cleaning	rinse	drying				
B					initial wash	detailed cleaning	rinse	drying

The total time it takes to process two cars is $5 \times 8 = 40$ minutes. So the time to clean one car is $40 / 2 = 20$ minutes.

The problem with this system is that, once car A has had the initial wash, that stage is then left idle, waiting for car A to complete, before car B enters. This is not efficient and, if we continue with this system, the only way we can improve the operation is to increase the speed of each stage.

It is the same situation with the performance of a CPU, where we are limited by the speed of the hardware, and improving this can be very expensive. Being more efficient with what we have is more beneficial.

In a pipelined solution, as soon as car A finishes a stage, car B enters that stage:

Car					
A	initial wash	detailed cleaning	rinse	drying	
B		initial wash	detailed cleaning	rinse	drying

The total time it takes to process two cars is $5 \times 5 = 25$ minutes. So the time to clean one car is $25 / 2 = 12.5$ minutes.

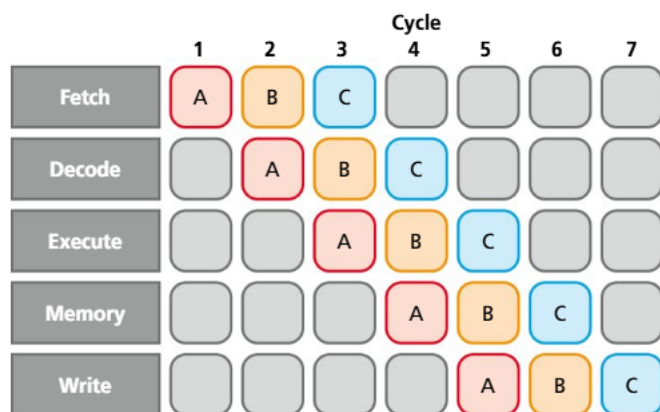
In this pipelined solution, rather than one stage sitting idle until the cycle is complete, the moment it is finished with car A, car B enters that stage.

■ Design of a basic pipeline

In a pipelined processor, the pipeline consists of multiple stages or segments situated between an input end and an output end. Each stage performs a specific operation, and the output of one stage becomes the input for the next. Intermediate outputs are held in interface registers, also known as “latches” or “buffers”. All stages and interface registers are synchronized by a common clock, ensuring co-ordinated operation across the entire pipeline.

In the CPU, the fetch–decode–execute cycle is divided into distinct stages:

- 1 **Fetch:** The instruction is retrieved from memory.
- 2 **Decode:** The instruction is interpreted to understand the required operation.
- 3 **Execute:** The operation is carried out.
- 4 **Memory access:** Any necessary data is read from or written to memory.
- 5 **Write back:** The result is written back to the CPU register.



■ Example of a pipeline cycle

Rather than measuring performance in minutes, as in the carwash example, pipeline performance in CPUs is measured in cycles. To manage the five stages mentioned, the CPU is constructed with a five-stage instruction pipeline, ensuring continuous and efficient processing of instructions. A well-optimized pipeline can achieve close to one instruction per cycle, maximizing the CPU's performance by reducing idle times and ensuring continuous instruction processing.

■ How cores in multi-core processors work independently and in parallel

In multi-core architectures, each core can independently execute its own pipeline of instructions. This is similar to having multiple carwash teams, each capable of processing cars simultaneously but independently. They are also capable of parallel execution when dealing with larger, more complex tasks, where each team completes a part of a larger task to improve execution time. This combination of pipelining and parallelism significantly boosts computational efficiency, enabling modern processors to handle complex and resource-intensive tasks more effectively.

Independent execution

Each core in a multi-core processor has its own set of pipelines, allowing it to fetch, decode, execute and write back instructions independently of the other cores. This independence means that, even if one core is handling a computationally intensive task, other cores can continue to execute their tasks without waiting for the first core to finish. This increases overall efficiency and utilization of the CPU resources.

Consider our carwash with multiple bays:

Team 1 (Core 1): Car A undergoes initial wash – detailed cleaning – rinse – drying

Team 2 (Core 2): Car B undergoes initial wash – detailed cleaning – rinse – drying

While Team 1 is drying car A, Team 2 might be rinsing car B. Both bays operate independently.

Parallel execution

Parallel execution takes the concept further, by allowing multiple cores to work on different parts of a single large task or multiple tasks simultaneously. For instance, in a multi-threaded application, different threads can be scheduled on different cores, with each core processing its thread in parallel. This drastically reduces the time needed to complete complex computations.

Imagine a large car that needs washing, detailing and interior cleaning. Multiple teams (cores) can work on different sections of the car at the same time:

Team 1 (Core 1): Washes the exterior

Team 2 (Core 2): Details the interior

Team 3 (Core 3): Cleans the wheels and undercarriage.

Each team works in parallel on different parts of the same car, drastically reducing the overall time required to complete the job.

Top tip!

Think of pipelining like an assembly line in a factory. Each stage in the pipeline handles a different part of the process and, once a stage finishes its task, it passes the work to the next stage and immediately starts on a new task. This way, multiple instructions are being processed simultaneously, just at different stages. In a multi-core architecture, imagine multiple assembly lines (cores) working in parallel, each running its own pipeline. This set-up greatly increases efficiency because more tasks are completed in less time, and the CPU can handle multiple instructions or even different programs at one time.

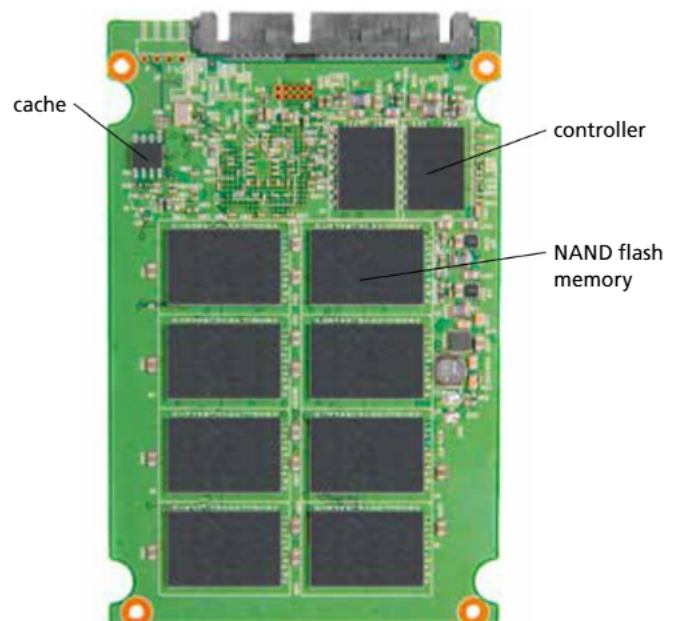
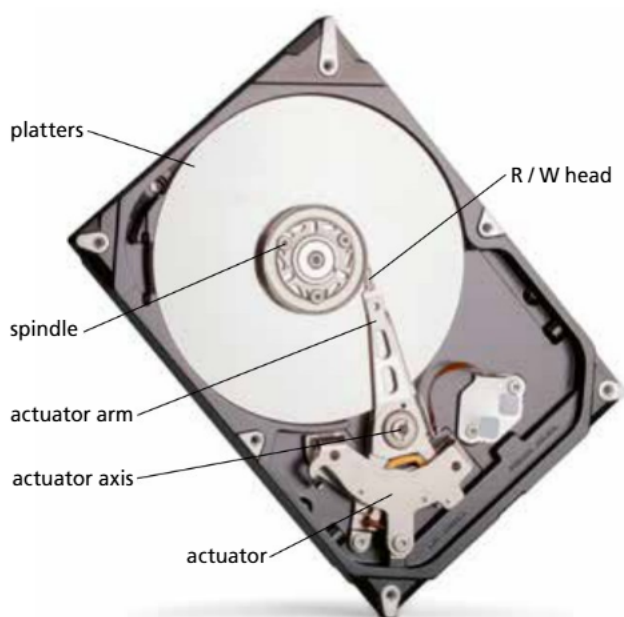
REVIEW QUESTIONS

- 1 What is pipelining and how does it improve performance?
- 2 How does a non-pipelined CPU differ from a pipelined CPU in terms of instruction execution?
- 3 What are the stages of a basic instruction pipeline, and how do they function together in a CPU?
- 4 How do multi-core processors use pipelining and parallel execution to improve computational efficiency?

A1.1.7 Internal and external types of secondary memory storage

Internal storage

Hard disk drive (HDD) and solid state drive (SSD)



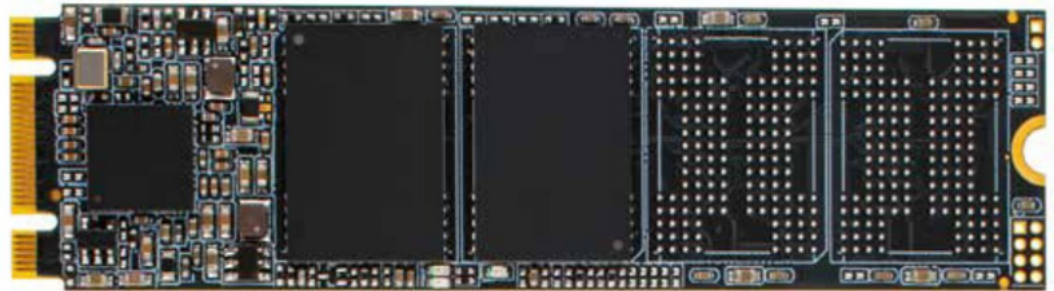
■ The internals of an HDD and an SSD

Hard disk drives (HDD) and solid state drives (SSD) are the most typical storage solutions for personal computers. HDDs are older technology but are still often used, especially in non-mobile devices, as they are relatively cheap compared to the amount of storage they offer. HDDs utilize a spinning magnetic disk to read / write data. They are suitable for storing large volumes of data, such as media files, backups and documents, where speed is not so critical. SSDs have no moving parts. They use flash memory to store data, offering high-speed data access and durability. This makes them very popular in portable devices such as laptops and tablets. They are ideal for operating systems, software applications and games due to their fast read / write speed, which enhances the overall system performance.

■ HDD vs SSD

Feature	HDD (hard disk drive)	SSD (solid state drive)
Storage technology	Magnetic storage with spinning disks and read / write heads	Flash memory with no moving parts
Speed	Slower read / write speeds (generally 50–150 MB/s)	Faster read / write speeds (generally 200–500 MB/s)
Durability	More prone to physical damage due to moving parts	More durable; resistant to physical shock
Noise	Produces noise due to moving parts	Silent operation
Power consumption	Higher power usage due to mechanical parts	Lower power consumption
Cost	Generally cheaper per GB	More expensive per GB
Capacity	Available in larger capacities (up to several TB)	Typically available in smaller capacities (up to several TB, but at a higher cost)
Weight	Heavier due to mechanical components	Lighter
Heat generation	Generates more heat due to moving parts	Generates less heat

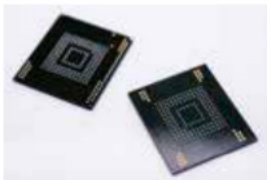
There is another form factor for SSDs that is currently popular and offers various advantages. M.2 SSDs look like a stick of chewing gum. They are very small and thin, and take up a lot less space than a standard SSD. M.2 NVMe SSDs are also faster than 2.5" SATA SSDs and are considered easier to install – you just slot them into the motherboard and use a single screw to keep them in place.



■ M.2 SSD

eMMC (Embedded MultiMediaCard)

In low-cost devices, such as entry-level smartphones and budget laptops, where all the benefits of SSDs are not essential, eMMCs are a popular choice. They are also a type of flash storage that utilizes NAND flash memory. They are soldered directly on to the motherboard of the device. While the capacity and speed do not match a standard SSD, their performance is adequate for basic computing needs and simple applications.



■ Two eMMCs



■ An external SSD

■ External storage

Hard disk drive (HDD) and solid state drive (SSD)

As external storage solutions, both HDD and SSD are popular choices. Their performance and comparison are identical to the internal versions. Which is used depends on the requirements of the user. If you require quick file transfers, backups and a portable solution that is less likely to be impacted by being carried around, SSDs are the best choice. If you need to do extensive backups, store media files or transport large files, but speed is less critical, you may decide an HDD is the better option.

Optical discs and optical drives



■ From left to right: CD, DVD and Blu-Ray

Optical drives that read / write optical discs, such as CDs, DVDs or Blu-Rays, are becoming less popular, but are still a consideration for external media storage. The cost of an optical disc is low compared to an HDD or SSD and, while their read / write speeds may be slower, they are sufficient for data archiving and playback. However, the discs are prone to scratches, especially if they are not stored correctly, and they require an optical drive to read and write to them, and these are becoming less common in devices these days.

Memory cards

Memory cards are compact storage devices often used in cameras, smartphones and other portable devices. They are ideal for expanding storage in mobile devices and for storing photos and videos in cameras, using NAND flash memory. They come in multiple sizes, such as SD, microSD and CompactFlash, catering to different devices and space requirements. They are known for their durability – they are resistant to physical shocks, extreme temperatures and water, making them ideal for portable devices. Their read / write times are generally slower than SSDs, but outperform those of optical discs.

◆ **RAID (Redundant Array of Independent Disks):** a data storage technology that combines multiple physical drives into a single logical unit to improve performance, provide redundancy and ensure data protection.

Network Attached Storage (NAS)

NAS is a dedicated file storage connected to a network that allows multiple users to access data. It is often used in homes or businesses for centralizing data storage, file storage and data backup. NAS is usually made up of multiple HDDs or SSDs configured in **RAID (Redundant Array of Independent Disks)** configuration. It is normally connected to the network via Ethernet, and runs a lightweight operating system designed for file storage, and the management and sharing of files. As it uses multiple HDDs or SSDs, its capacity is usually high, and it is possible to expand the system further by adding additional drives.



■ Memory cards



■ NAS storage solution

REVIEW QUESTIONS

- 1 What are the primary differences between an HDD and an SSD in terms of performance and durability?
- 2 Why might a low-cost device, such as an entry-level smartphone, use eMMC storage instead of an SSD?
- 3 What advantages do NAS (Network Attached Storage) systems offer for home or business environments?
- 4 How do memory cards compare to optical discs in terms of durability and data storage capabilities?

A1.1.8 Describe the concept of compression

Compression is the process of encoding information using fewer bits than the original representation. Making file sizes smaller has two main advantages: it takes less room on secondary storage and it is faster to transfer across a network. There are two main types of compression: lossless and lossy.

■ Lossless vs lossy compression

Lossless compression is when data is compressed to a smaller size, but can be restored back to the original without any loss of information. This is important for files such as text files and databases, where a loss of information would be critical. This technique works by identifying and eliminating **statistical redundancy** within the data, and this process can be reversed when needed.

Lossy compression generally outperforms lossless compression when it comes to file sizes; however, it reduces files by permanently eliminating certain information. This information is redundant or less critical data, resulting in a compressed version that is not identical to the original but is, ideally, indistinguishable from the original to human senses. Lossy compression is commonly used for compressing multimedia files such as images, audio and video, where some loss of quality is acceptable in exchange for significantly reduced file sizes.

◆ **Statistical redundancy:** the repetition of information within a data set that does not contribute to its uniqueness.

This can be seen in the images below. While it may be pretty difficult to visually distinguish the difference in quality, the lossy version uses 50 per cent less data than the original.



Original	Lossless	Saved	Lossy	Saved
1.73 MB	1.58 MB	9%	886 KB	50%

■ Run-length encoding (RLE)

Run-length encoding is an effective lossless data-compression technique used to reduce the size of files containing many consecutive repeated characters.

For example, take this string:

AAAAABBBCCDAA

RLE looks for “runs” where a character is repeated. In the example above, we have five runs:

AAAAA BBB CC D AA

Once RLE has identified these, it encodes the run by replacing it with a pair: the character that repeats and the number of repetitions. So, the runs above become:

5A 3B 2C 1D 2A

The encoded string is then stored as:

5A3B2C1D2A

If we assume each letter stores 8 bits of information, the initial data is $13 \times 8 = 104$ bits, or 13 bytes.

After compressing with RLE, the data is $10 \times 8 = 80$ bits or 10 bytes: a 23 per cent reduction in size.

RLE is straightforward to implement and it is very effective for data with lots of repetitions, such as simple graphics and certain types of text files. RLE was often used on fax machines, which would send text documents via the telephone line. This was because they contained a lot of white space, which meant RLE could achieve compression ratios of up to 8:1. However, for data that does not contain many repeated characters, like a portrait photograph, RLE may not be very effective and, in some cases, may even increase the file size.

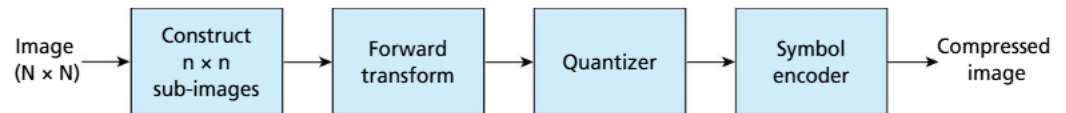
PROGRAMMING EXERCISE

Create an RLE application that has two options: compress or decompress.

The compress option should receive a string and output the encoded version using the RLE algorithm.

The decompress option should do the opposite.

■ Transform coding



■ The stages of transform coding

Transform coding is a form of lossy compression often used in JPEG image compression or MP3 audio compression.

Using JPEG compression as an example:

- Transform coding takes an image of $N \times N$ size and sections it into smaller sub-images of size $n \times n$.
- Then the *forward transform* is carried out on each of the sub-images. The forward transform can use different algorithms, depending on the type of file compression, but for JPEGs DCT (discrete cosine transform) is used. This takes the image data from the spatial domain (pixel values) to the frequency domain. The output breaks the sub-image down into **low-** and **high-frequency** coefficients.
- These frequency coefficients are then passed to the *quantizer*. This step significantly reduces file size by simplifying the frequency coefficients obtained from the DCT. The purpose of quantization is to reduce the precision of high-frequency components (the fine details) rather than low-frequency components. This is because the human eye is less sensitive to high-frequency data loss compared to low-frequency detail. The extent of the quantization determines the compression level and the quality of the final image.
- The final step of transform coding is the *symbol encoder*. This is where the quantized coefficients are further compressed using entropy coding techniques. This runs through three further algorithms to reduce the file size by efficiently representing the frequency of occurrence of each symbol. The algorithms used at this stage are (in this order):
 - 1 Zigzag scan
 - 2 Run-length encoding (RLE)
 - 3 Huffman coding.

Once this stage has finished, the final compressed image is complete.

◆ **Low-frequency data:** correspond to slow changes in pixel values, such as broad areas.

◆ **High-frequency data:** correspond to rapid changes in pixel values, representing fine details, edges and textures.

REVIEW QUESTIONS

- 1 What are the two main advantages of compressing files?
- 2 Explain the difference between lossless and lossy compression.
- 3 How does run-length encoding (RLE) work, and in what types of files is it most effective?
- 4 Describe the process of transform coding in JPEG image compression and explain why it is considered a lossy compression method.

A1.1.9 Types of services in cloud implementation

Cloud computing has revolutionized how organizations manage and deploy IT resources, offering flexible and scalable solutions to meet diverse business needs. There are three primary cloud service models: Software as a Service (SaaS), Platform as a Service (PaaS) and Infrastructure as a Service (IaaS). Each one provides distinct levels of control, flexibility and management solutions.

■ Software as a Solution (SaaS)

SaaS delivers software applications over the internet. Users can access these applications through web browsers without needing to install, maintain or update the software locally. SaaS provides a cost-effective and convenient solution for businesses and individuals, offering a wide range of applications from productivity tools to customer-relationship management systems.

SaaS allows users to access their software from anywhere, on any device, as long as they have an internet connection. This eliminates the need for complex software installations. Many SaaS providers charge a subscription fee, which is often less than the cost of purchasing software licences. Additionally, updates and new features are automatically added by the provider, ensuring that users always have the most up-to-date version of the software.

However, SaaS software relies on the user having an internet connection; without it, they cannot run the software, unlike locally installed software. Data security is also a concern, as users rely on the provider's security measures to protect sensitive data.

Example

Google Workspace is an example of SaaS. This suite provides productivity tools, including Gmail, Google Docs and Google Drive, used by businesses and educational institutions for communication, collaboration and storage.

■ Platform as a Service (PaaS)

PaaS provides a cloud-based platform that allows developers to build, test and deploy applications without managing the underlying infrastructure. PaaS includes tools and services to facilitate application development, such as databases, **middleware** and development frameworks.

PaaS accelerates software development by allowing developers to focus on coding rather than infrastructure management. It also makes it easier and cheaper to scale hardware as the user base increases. However, this solution can lead to vendor lock-in, making it difficult to move applications to different platforms and offering less control over the hosting environment.

Example

Microsoft Azure App Service is an example of PaaS. It is a platform for building, deploying and scaling web apps and APIs, used by developers to create scalable and reliable applications without managing the underlying servers.

■ Infrastructure as a Service (IaaS)

IaaS provides virtualized computing resources over the internet, such as virtual machines, storage and networks. This allows businesses to rent IT infrastructure instead of buying and managing physical servers.

Unlike PaaS, IaaS gives users full control over their virtual machines and networks. This reduces the need for upfront investment in hardware and allows businesses to rent solutions at a lower initial cost using a subscription model. IaaS is also scalable, making it easy to adjust resources as the user base grows. However, IaaS requires more technical knowledge than PaaS, as users must manage their own devices and secure their own data and applications.

Example

Amazon Web Services (AWS) EC2 is an example of IaaS. Businesses use AWS EC2 to create and manage virtual servers, providing the flexibility to run applications without owning physical hardware.

◆ **Middleware:**

software that connects different applications, allowing them to communicate and share data. It helps different parts of a computer system work together smoothly.

REVIEW QUESTIONS

- 1 What is Software as a Service (SaaS) and how does it differ from traditional software installation?
- 2 Explain how Platform as a Service (PaaS) benefits software developers.
- 3 Why might a business choose Infrastructure as a Service (IaaS) over purchasing physical hardware?

EXAM PRACTICE QUESTIONS

Note: All the exam practice questions are representative of those that will be found on Paper 1 for the International Baccalaureate Diploma in Computer Science.

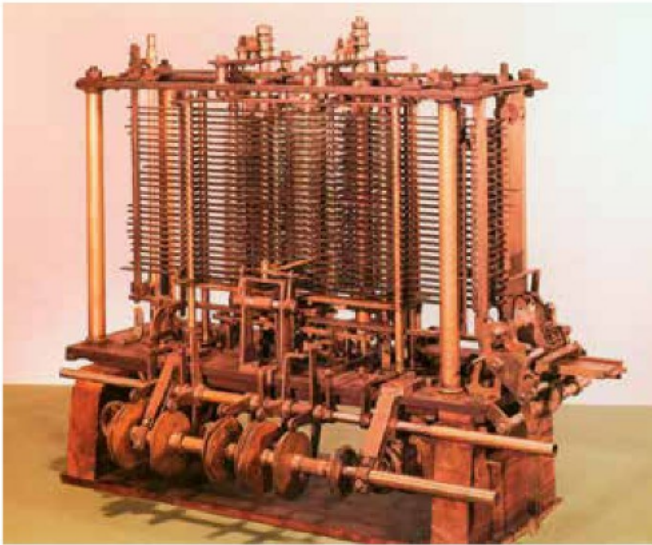
- 1 Describe the function of the arithmetic logic unit (ALU). [2]
- 2 Outline the role of the program counter (PC). [2]
- 3 Explain the advantages of multi-core processors compared to single-core processors. [3]
- 4 Describe how the architecture of a GPU differs from a CPU, and why it is better suited for tasks such as video rendering. [3]
- 5 Compare the processing power of a CPU and a GPU in handling complex computations. [4]
- 6 Explain the role of L1 cache in a computer system. [2]
- 7 Describe the fetch–decode–execute cycle that a CPU uses to process instructions. [4]
- 8 Explain the concept of pipelining in multi-core processors. [3]
- 9 Describe the differences between solid state drives (SSD) and hard disk drives (HDD). [4]
- 10 Describe the method of lossy compression and give an example of its use. [3]

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A1.2.1 Describe the principal methods of representing data
- ▶ A1.2.2 Explain how binary is used to store data
- ▶ A1.2.3 Describe the purpose and use of logic gates
- ▶ A1.2.4 Construct and analyse truth tables
- ▶ A1.2.5 Construct logic diagrams

A1.2.1 Principal methods of representing data



■ The Analytical Engine, conceived by Charles Babbage in the 19th century



■ The Setun computer, developed in 1958

Binary (base-2) is the language for modern-day computers; however, this was not always the case. When developing early computers, several number systems were trialled. Charles Babbage, the inventor of the Analytical Engine, used decimal for his inventions. This seemed a logical choice as people already commonly used base-10.

The ternary system (base-3) was also explored. The Setun computer, developed in the Soviet Union in 1958, used this system. Over 50 of these were produced for educational and scientific institutions to help explore the benefits of ternary logic in computing. Despite its innovative approach, the practical challenges and the widespread adoption of binary logic eventually led to its replacement.

Other scientists and inventors also explored quaternary (base-4) and other number systems. However, practical implementations of these systems were rare due to the increased complexity in hardware design and the limited benefits compared to binary.

Modern computing ultimately settled on binary (base-2) as the primary number system. The base-2 system represents two possible states: 1 or 0. This is in contrast to the number system we are all comfortable with, the decimal system (base-10), which has ten possible states: 0 to 9. The binary system is particularly well-suited to represent the state of electrical switches within a computer system: on (1) and off (0). This simplicity reduces hardware complexity and enhances reliability.

Binary reduces the complexity in hardware design because digital electronics, such as transistors, naturally operate in binary mode. Transistors act as switches that can be